Technical RETEX

Automating timeline reconstruction and native Text+ creation in DaVinci Resolve

Use case: a timeline generated with Remotion Studio

Summary

This experience report describes the solution used to reconstruct in DaVinci Resolve a timeline produced outside Resolve, with particular focus on automated creation of native, editable Text+ titles.

The complete bridge is specific to IMS, an in-house application for automated video editing built around Remotion Studio, and is not published here.

The reusable mechanisms are isolated below: a Text+ source in the Media Pool, a positioning carrier, replacement by a MediaPoolItem, frame-rate consistency, and a bridge running inside Resolve Free.

Primary test environment: Windows, DaVinci Resolve 20.3.1 Build 4 and 21.0.2 Build 4.

Native editable Text+: a reproducible solution

Problem

A reconstructed timeline can receive media and timing information without major difficulty, while automatically creating a true editable Text+ title is more problematic. A title created directly on the timeline yields a TimelineItem; the method used here requires a Text+ source available in the Media Pool as a MediaPoolItem.

Solution

The working mechanism uses a native Text+ source in the Media Pool and a temporary clip on the timeline. In this document that temporary clip is called a carrier. It is only a placeholder: it carries the intended start position and duration of the future title.

- Create or import a genuine native Text+ source into the Media Pool.
- Create the carrier at the exact start position and duration required for the title.
- Identify the MediaPoolItem corresponding to the Text+ source.
- Replace the carrier with that source while preserving its timeline position.
- Access the TextPlus/Fusion node of the new item and apply the required text and parameters.
- Verify the resulting object type, start position and duration.

carrier → Media Pool Text+ source → replacement → TextPlus parameters

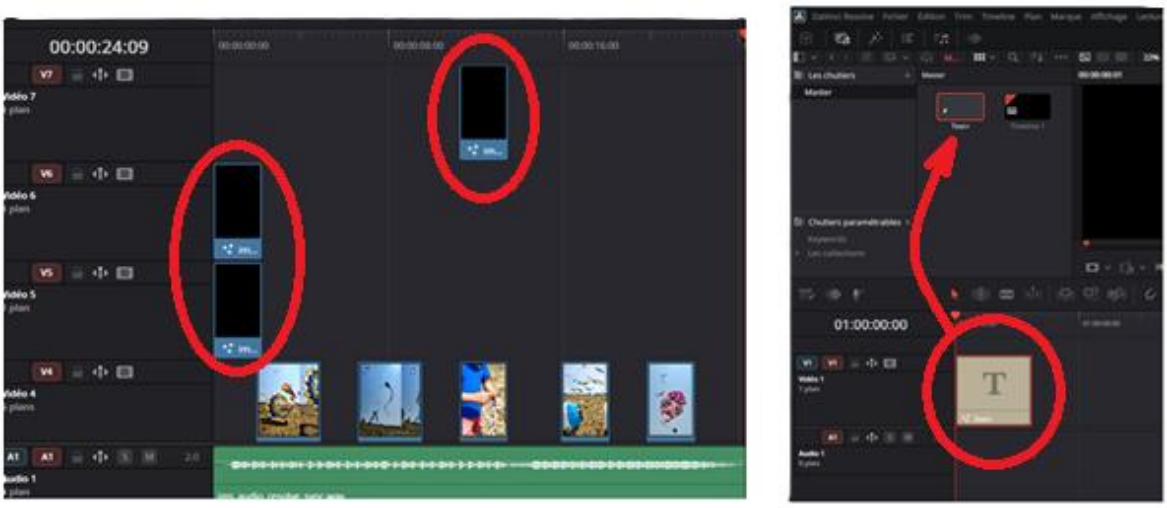

Figure 1 - Left: carriers define position and duration. Right: a Text+ source is available in the Media Pool and the resulting title remains native and editable.

In the tested bridge, the Text+ source is carried in a Resolve bin (.drb) imported into the Media Pool. The .drb itself is not a conceptual requirement: the essential point is to have a genuine Text+ source available as a MediaPoolItem.

Community projects use related approaches: AutoSubs distributes a caption-bin.drb containing a Text+ clip, while Resolve OpenCaptions relies on Text+ templates placed in the Media Pool.

If the Text+ duration is wrong: check the source frame rate

Observed symptom: in a 25 fps timeline, a Text+ source prepared at 24 fps produced durations slightly longer than requested.

Requested duration	Observed duration
57 frames	~59
75 frames	~78
100 frames	~104

The discrepancy is close to the 25/24 ratio. The cause was neither Remotion nor the Remotion -> Resolve exchange: the Text+ source and the target timeline were using different time bases.

Solution: make the Text+ source consistent with the target timeline frame rate before using it. The final bridge adapts the resource to the frame rate carried by the timeline description; no arbitrary duration compensation is required.

Controlling Resolve Free from an external process

Problem

In the tested Resolve Free environment, the external process was not the appropriate context for directly executing the required Resolve operations.

Solution

The bridge runs inside Resolve. The external application writes a command to a channel accessible by both processes; the bridge reads it, performs the operation through the Resolve API, and writes the response.

external application → request → bridge inside Resolve → Resolve API → response

In the tested implementation, communication uses request/response files. An independent open-source implementation, Mateo Khalil's davinci-resolve-mcp, also uses a Lua bridge running inside Resolve Free with file-based communication.

A communication timeout must be distinguished from a Resolve failure: the operation may have completed even if its acknowledgement did not arrive within the expected time. Before retrying, check the state actually produced in Resolve.

Carry the timeline context with the edit description

The reconstruction used here relies on a reference description of the edit that is independent of Resolve. It carries media, positions, durations, transforms and text, together with the target timeline dimensions and frame rate.

Remotion Studio → reference edit description → DaVinci Resolve bridge → Resolve timeline

The bridge therefore builds the timeline in the requested context instead of assuming a fixed format or frame rate. The same context is used to prepare dependent resources, including the Text+ source.

The use case described here starts from Remotion Studio, but the mechanism is not tied to Remotion: any application capable of producing an equivalent description can feed the same type of reconstruction.

Code example published with this RETEX

A minimal extract of the validated mechanism is provided separately as `textplus_example.py`. It isolates the carrier -> MediaPoolItem Text+ -> native insertion -> Fusion parameter copy sequence without publishing IMS orchestration or the complete bridge. Creation/adaptation of the .drb source is intentionally outside the example.

Resources worth checking before rebuilding the same mechanisms

Resource	Why it is relevant
AutoSubs - Thomas Moroney	Example of a Text+ source distributed through a Resolve bin.
Resolve OpenCaptions - David C.	Generation of Text+ titles from templates stored in the Media Pool.
davinci-resolve-mcp - Mateo Khalil	Internal bridge for Resolve Free using file-based request/response exchange.
OpenTimelineIO (OTIO)	Standardized model for editorial interchange that may replace part of a custom JSON representation.
DaVinci Resolve Scripting API	Reference for the functions officially exposed by Resolve.

OTIO was not validated as a replacement for the IMS bridge. It may reduce the amount of application-specific editorial representation, but Resolve-specific elements may still require an additional layer.

Key points

- To obtain a native editable Text+, use a genuine Text+ source in the Media Pool and replace a carrier positioned on the timeline.
- If title duration drifts, first check the Text+ source frame rate against the target timeline frame rate.
- In the tested Resolve Free environment, running the bridge inside Resolve allows an external application to drive the required operations.
- Timeline dimensions and frame rate should travel with the edit description so reconstruction and dependent resources use the same context.

Technical references

Academy Software Foundation - OpenTimelineIO
API and editorial interchange format; timeline model, adapters and conversion limitations.

Blackmagic Design - DaVinci Resolve Scripting API
Reference documentation for the functions used to materialize the timeline.

Thomas Moroney - AutoSubs
Resolve integration and caption-bin.drb containing the Fusion Text clip.

David C. - Resolve OpenCaptions
Creation of Text+ tracks from Media Pool templates.

Mateo Khalil - davinci-resolve-mcp
Bridge for Resolve Free running inside Resolve with file-based communication.

Experimental results

The carrier/Text+ mechanism, the observed 24/25 fps timing behavior, and propagation of timeline dimensions/frame rate come from the IMS/DaVinci Resolve experiments. They describe behavior observed in the tested environments and are not presented as general guarantees from Blackmagic Design.